

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a\agent-a2a04340244432509.jsonl`
- SHA-256: `6989b68257237fa2703178f2b4441f0171c1960db5915e231b26b0e0d231cd6a`
- Source modified: `2026-07-06T07:58:56+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:57:10.188Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:

- db.py: `_migration_004_destinations_and_routing` (backfills destinations from `filter_config_deprecated`, `routing_rules` per source, `delivery_attempts` from `videos_deprecated.forwarded=1`); `repos` `upsert_destination`, `get_destination`, `upsert_delivery_attempt`, `record_delivery` (UPSERT with `attempt_count` increment), `get_delivery_attempt`.
- delivery.py: `forward_to_target/send_link_message/deliver` gain an optional `destination_id` kwarg; when set, `record_delivery` writes a `DeliveryAttempt`; when None (all current live callers) falls back to the `mark_forwarded` no-op.
- models.py: `Destination`, `RoutingRule`, `DeliveryStatus`, `DeliveryAttempt`.

Note: routing is NOT yet wired into the live path (`telethon_client/manager` still call `deliver()` with `destination_id=None`); `destination_id` is currently exercised only by tests. So `record_delivery` issues are LATENT until Track D, but ship now.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: `C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>`. Build schema via `db._apply_schema(conn)` on `aiosqlite ":memory:"`; or `db.init_db(path)` on a temp file seeded with a legacy v1 schema (`users/channels/videos/filter_config`) to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns `duration_s,width,height,size_bytes,mime_type` or migration 002's column-copy fails (that's a test-harness requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): `ruff check clean`; `pytest 451` passed @ 95.75% (floor 80%); `ruff format --check flags commands.py` ONLY because this branch predates the merged #36 format fix (`commands.py` is NOT in this PR) ? do NOT report that as a #37 finding.

- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from LATENT (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding. Rank severe first.

- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug, narrow trigger), medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic triggers.

ADVERSARIALLY VERIFY this single finding ? try to REFUTE it. Read the exact cited code, trace real control flow, and run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour (note live vs latent). REFUTED if the code handles it, the trigger can't occur, or it's pre-existing/not in this diff. PLAUSIBLE if you can't fully settle it. Give corrected severity.

FINDING (dim migration_004):
title: Account with legacy sources/forwarded videos but no filter_config gets zero destinations/routes/history
severity(claimed): low | latent
where: src/tg_video_filter/db.py:702
summary: All three backfills are driven off filter_config_deprecated; an account lacking a filter_config row produces 0 destinations, 0 routing_rules, and 0 delivery_attempts, silently dropping its forwarded-delivery history and leaving it with no delivery path.
failure_scenario: A legacy account that has channels + forwarded videos but no filter_config row gets nothing backfilled: no destination, no route, and its previously-forwarded videos are not recorded as delivered (so a future re-scan could re-forward them). Reproduced: omitting the filter_config row yields destinations/routing_rules/delivery_attempts all 0 while content_items is still populated (scenario C).
evidence: db.py:702 (step 1) and the routing_rules subquery (db.py:720) both FROM filter_config_deprecated; no fallback destination is created when the row is absent. Reachability is very low: bootstrap.py:75 always persists DEFAULT_FILTER_CONFIG for every onboarded account, and an account cannot accumulate forwarded videos without having run a bootstrapped client. Hence low.
recommendation: Optionally add a fallback that creates a default 'me' destination for any account that has sources but no filter_config_deprecated row, so no legacy account is left routeless and its forwarded history is still recorded; or document that this state is structurally impossible post-bootstrap.

2. assistant (2026-07-06T07:57:14.265Z)

I'll verify this finding by reading the actual migration code and tracing the control flow.

3. user (2026-07-06T07:57:15.539Z)

```
670
671     Creates a default destination row for each account that has a
672     ``filter_config_deprecated.delivery_target`` value, then creates a default
673     routing rule (source ? default policy ? destination) for each source.
674     Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675     table (one ``delivered`` attempt per previously-forwarded video to the
676     account's default destination).
677
678     Idempotent: skips accounts whose default destination already exists.
679     """
680     # Gate: if destinations already have data, this migration has run.
681     async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682         if (await cur.fetchone())[0] > 0:
683             return
684
685     if not await _table_exists(conn, "filter_config_deprecated"):
686         return
687
```

```

688     # 1. Backfill default destinations from filter_config_deprecated.
689     #     Parse the JSON delivery_target, defaulting to "me".
690     await conn.execute(
691         """
692         INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
693         SELECT user_id, 'telegram',
694             COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
695             CASE
696                 WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
697                 THEN 'saved'
698                 ELSE 'channel'
699             END,
700             0,
701             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
702         FROM filter_config_deprecated
703         """
704     )
705
706     # 2. Create a default routing rule per source (source ? default policy ? default
destination).
707     await conn.execute(
708         """
709         INSERT OR IGNORE INTO routing_rules
710             (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
711         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
712             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
713         FROM sources s
714         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
715         JOIN destinations d ON d.account_id = s.account_id
716         WHERE d.target = (
717             SELECT COALESCE(
718                 json_extract(fcd.config_json, '$.delivery_target'), 'me'
719             )
720         FROM filter_config_deprecated fcd
721         WHERE fcd.user_id = s.account_id
722         LIMIT 1
723         )
724         """
725     )
726
727     # 3. Backfill delivery_attempts from previously-forwarded videos.
728     await conn.execute(
729         """
730         INSERT OR IGNORE INTO delivery_attempts
731             (content_item_id, destination_id, route_id, status, attempt_count,
732             created_at, delivered_at)
733         SELECT ci.id, d.id, rr.id, 'delivered', 1,
734             vd.seen_at, vd.seen_at
735         FROM videos_deprecated vd
736         JOIN content_items ci
737             ON ci.account_id = vd.user_id
738             AND ci.provider = 'telegram'
739             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
740         JOIN sources sc
741             ON sc.id = ci.source_id
742             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
743         JOIN destinations d ON d.account_id = vd.user_id
744         JOIN routing_rules rr
745             ON rr.account_id = vd.user_id
746             AND rr.source_id = sc.id
747             AND rr.destination_id = d.id
748         WHERE vd.forwarded = 1

```

```

749         """
750     )
751
752
753     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
754         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
755         async with conn.execute(
756             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
757         ) as cur:
758             return await cur.fetchone() is not None
759
760
761     # --- helpers -----
762
763
764     def _now() -> datetime:
765         """Timezone-aware UTC 'now', used for policy timestamps."""
766         return datetime.now(UTC)
767
768
769     def _safe_int_cast(value: object, field_name: str) -> int:
770         """Cast a provider_source_id / provider_message_id / document_id to int.
771
772         All existing data is Telegram ? these ids are numeric strings. Non-Telegram
773         providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
774         shapes (which carry Telegram-specific int ids); this function raises a clear
775         ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
776         (ADR 0011 provider-neutrality guard ? review finding #5).
777         """
778         try:
779             return int(value) # type: ignore[arg-type]
780         except (ValueError, TypeError) as exc:
781             raise ValueError(
782                 f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
783
784                 f"types are Telegram-specific; use ContentItem/Source directly for "
785                 f"non-Telegram providers (ADR 0011)."
786             ) from exc
787
788     def _row_to_account(row: aiosqlite.Row) -> Account:
789         return Account(

```

4. assistant (2026-07-06T07:57:21.581Z)

The finding hinges on step 1 being driven off ``filter_config_deprecated``. Let me confirm the migration ladder ordering and the schema, plus check the reachability claim in bootstrap.py.

5. user (2026-07-06T07:57:24.249Z)

```

655         vc,
656         cic,
657         vc - cic,
658     )
659
660     # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661     #     rollback safety net (ADR 0010 pattern).
662     await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663     await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
664     await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
665
666
667     @_migration(4)
668     async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).

```

```

670
671     Creates a default destination row for each account that has a
672     ``filter_config_deprecated.delivery_target`` value, then creates a default
673     routing rule (source ? default policy ? destination) for each source.
674     Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675     table (one ``delivered`` attempt per previously-forwarded video to the
676     account's default destination).
677
678     Idempotent: skips accounts whose default destination already exists.
679     """
680     # Gate: if destinations already have data, this migration has run.
681     async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682         if (await cur.fetchone())[0] > 0:
683             return
684

```

6. assistant (2026-07-06T07:57:33.235Z)

Now let me verify the reachability claim in bootstrap.py.

7. user (2026-07-06T07:57:34.099Z)

```

55
56     Pure DB operation ? unit-tested. Returns the persisted user (legacy view).
57
58     ADR 0010: onboarding now writes one row to ``accounts`` (the product
59     identity) and one to ``telegram_sessions`` (the MTProto credential), rather
60     than a single conflated ``users`` row. The legacy ``User`` shape is
61     preserved as the return type so existing callers/tests are unaffected.
62     """
63     async with aiosqlite.connect(db_path) as conn:
64         await db._apply_schema(conn)
65         await db.upsert_account(conn, Account(id=user.id, telegram_id=user.id))
66         await db.upsert_telegram_session(
67             conn,
68             TelegramSession(
69                 account_id=user.id,
70                 phone=user.phone,
71                 session_path=f"user_{user.id}.session",
72                 enabled=user.enabled,
73             ),
74         )
75         await db.save_filter_config(conn, user.id, DEFAULT_FILTER_CONFIG)
76         persisted = await db.get_user(conn, user.id)
77         assert persisted is not None # just inserted
78         return persisted
79
80
81     async def bootstrap( # pragma: no cover - interactive Telethon login (real connection +
human)
82         settings: Settings,
83         *,
84         phone_prompt: PhonePrompt = _default_phone_prompt,
85         code_prompt: CodePrompt = _default_code_prompt,
86         password_prompt: PasswordPrompt = _default_password_prompt,
87     ) -> User:
88         """Run the interactive onboarding for one user.
89
90         1. Ensure the sessions dir exists.
91         2. Build a Telethon client with a file session at ``user_<pending>.session``.
92         3. ``client.start()`` with the prompt callables (interactive login).
93         4. ``get_me()`` to read the authorized user id.
94         5. Rename the session file to ``user_<id>.session`` (so the manager finds it).

```

8. assistant (2026-07-06T07:57:40.021Z)

The finding's own evidence confirms the reachability argument: `bootstrap.py:75` always persists `DEFAULT\_FILTER\_CONFIG` for every onboarded account via `save\_filter\_config`. Let me verify that `save\_filter\_config` writes to the `filter\_config` table (which becomes `filter\_config\_deprecated`) and that there's genuinely no other path to accumulate forwarded videos.

9. user (2026-07-06T07:57:42.204Z)

```
81:CREATE TABLE IF NOT EXISTS filter_config (  
207:  # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for  
215:      await conn.execute("DROP TABLE IF EXISTS filter_config")  
357:  data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,  
363:  FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with  
425:  # Rebuild channels / videos / filter_config so their FK constraints point  
472:      "filter_config",  
474:      CREATE TABLE filter_config (  
499:  # filter_config tables were already rebuilt above with FKs to accounts(id),  
512:  ``channels`` / ``videos`` / ``filter_config`` tables and renames the  
617:  # 3. Backfill policies from filter_config: one 'default' policy per account  
618:  #      (Option 2A). is_default=1 so the legacy load_filter_config shim reads  
627:      FROM filter_config  
664:  await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")  
1662:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:  
1665:  ADR 0011 (Option 2A): ``filter_config`` is renamed to  
1675:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)  
-> None:  
1771:  "load_filter_config",  
1772:  "save_filter_config",
```

10. assistant (2026-07-06T07:57:54.450Z)

The finding is essentially a self-refuting "low/latent" note. The claim that step 1 drives off `filter\_config\_deprecated` and produces zero rows when the row is absent is factually correct (db.py:702, subquery at db.py:720). But the finding itself demonstrates the trigger is structurally unreachable in practice: every onboarded account gets a `filter\_config` row via `bootstrap.py:75`, and an account cannot accumulate forwarded videos without a bootstrapped client.

Let me run a quick repro to confirm the mechanical behavior (that omitting filter_config yields zero rows), just to settle whether the described behavior actually occurs.

11. user (2026-07-06T07:57:55.460Z)

```
600      'tg:fp:' || size_bytes || ':' || duration_s || ':'  
601      || height || ':' || width  
602  WHERE document_id IS NULL  
603      AND size_bytes IS NOT NULL AND duration_s IS NOT NULL  
604      AND height IS NOT NULL AND width IS NOT NULL  
605      AND dedup_key LIKE 'tg:msg: %'  
606      AND id IN (  
607          SELECT MIN(id) FROM content_items  
608          WHERE document_id IS NULL  
609              AND size_bytes IS NOT NULL AND duration_s IS NOT NULL  
610              AND height IS NOT NULL AND width IS NOT NULL  
611              AND dedup_key LIKE 'tg:msg: %'  
612          GROUP BY account_id, size_bytes, duration_s, height, width  
613      )  
614  """"  
615  )  
616  
617  # 3. Backfill policies from filter_config: one 'default' policy per account  
618  #      (Option 2A). is_default=1 so the legacy load_filter_config shim reads
```

```

619         # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620         await conn.execute(
621             """
622             INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
623                                             created_at, updated_at)
624             SELECT user_id, 'default', config_json, 1,
625                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
626                    strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
627             FROM filter_config
628             """
629         )
630
631         # 4. Create the policies default-partial index (sig after backfill so the
632         # UNIQUE constraint only applies once rows are populated). On fresh DBs
633         # _SCHEMA creates it; on migrated DBs this is the first time.
634         await conn.execute(
635             "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
636             "ON policies(account_id) WHERE is_default = 1"
637         )
638
639         # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
640         # produce a content_items row. A mismatch signals fingerprint collisions
641         # where two distinct videos shared (size,dur,h,w) ? the losers stay at
642         # tg:msg: keys and are still present. Log a warning so the operator can
643         # investigate; the data is preserved in *_deprecated tables.
644         async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
645             vc = (await cur.fetchone())[0]
646         async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
647             cic = (await cur.fetchone())[0]
648         if vc != cic:
649             import logging
650
651             _logger = logging.getLogger(__name__)
652             _logger.warning(
653                 "migration_003: %d videos rows produced %d content_items rows "
654                 "%d gap). Colliding items remain in videos_deprecated.",

```

12. assistant (2026-07-06T07:57:59.909Z)

This confirms a key related fact: migration 003 (step 3, db.py:620-629) also backfills the default **policy** off `filter_config`. So an account with no `filter_config` row gets no default policy either. This is relevant because step 2 of migration 004 (routing_rules) requires `JOIN policies p ON p.is_default = 1` (db.py:714) ? so even if a destination existed, no default policy would mean no routing rule.

Now let me run the repro to mechanically confirm the described behavior.

13. user (2026-07-06T07:58:10.796Z)

Exit code 1

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\repro37.py", line 37, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()

```

```

~~~~~^^
File "C:\Users\avidu\AppData\Local\Temp\repro37.py", line 30, in main
    await db.init_db(path)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 196, in init_db
    await _apply_schema(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 216, in _apply_schema
    await _apply_migrations(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 324, in
_apply_migrations
    await fn(conn)
File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 391, in
_migration_002_identity_split
    await conn.execute(
    ...<4 lines>...
    )
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
    ^^^^^^^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.OperationalError: no such column: created_at

```

14. assistant (2026-07-06T07:58:14.470Z)

Migration 002 needs more legacy columns. Let me check what migration 002 expects from the users/channels tables.

15. user (2026-07-06T07:58:15.157Z)

```

385         provider      TEXT NOT NULL DEFAULT 'telegram',
386         display_name TEXT,
387         created_at     TEXT NOT NULL
388     )
389     """"
390 )
391 await conn.execute(
392     """"
393     INSERT INTO accounts (id, telegram_id, provider, created_at)
394     SELECT id, id, 'telegram', created_at FROM users
395     """"
396 )
397
398 # Build telegram_sessions, one row per legacy userbot user.
399 # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
400 await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
401 await conn.execute(
402     """"
403     CREATE TABLE telegram_sessions (
404         account_id     INTEGER PRIMARY KEY REFERENCES accounts(id),
405         phone          TEXT,
406         session_path   TEXT NOT NULL,
407         enabled        INTEGER NOT NULL DEFAULT 1,
408         created_at     TEXT NOT NULL,
409         last_login_at  TEXT
410     )
411     """"
412 )
413 # session_path is built from the *telegram_id* (= old users.id) to match
414 # the convention telethon_client.build_client used pre-migration. The

```

```

415     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
416     # so this migration does not need to know the runtime path.
417     await conn.execute(
418         """
419         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
420         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
421         FROM users
422         """
423     )
424
425     # Rebuild channels / videos / filter_config so their FK constraints point
426     # at accounts(id) instead of the legacy users(id). SQLite's
427     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
428     # the rename ? so a naive rename of users ? users_deprecated would leave
429     # these tables referencing users_deprecated(id), diverging from fresh DBs

```

16. assistant (2026-07-06T07:58:17.619Z)

Legacy `users` needs `created\_at`, and `channels`/`videos` likely need more columns too. Let me look at an existing test's legacy schema fixture to build a valid one.

17. user (2026-07-06T07:58:19.268Z)

```

646: CREATE TABLE users (
647:     id INTEGER PRIMARY KEY, phone TEXT,
648:     enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
649: );
650: CREATE TABLE channels (
651:     id INTEGER PRIMARY KEY AUTOINCREMENT,
652:     user_id INTEGER NOT NULL REFERENCES users(id),
653:     channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
654:     UNIQUE (user_id, channel_id)
655: );
656: CREATE TABLE videos (
657:     id INTEGER PRIMARY KEY AUTOINCREMENT,
658:     user_id INTEGER NOT NULL REFERENCES users(id),
659:     channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
660:     duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
661:     mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
662:     forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
663:     UNIQUE (user_id, channel_id, message_id)
664: );
665: CREATE TABLE filter_config (
666:     user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
667: );
668: """
669: async with aiosqlite.connect(db_path) as conn:
670:     await conn.executescript(legacy_schema)
671:     # Seed rows so we can confirm data survives the migration.
672:     await conn.execute(
673:         "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z')"
674:     )
675:     await conn.execute(
676:         "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
677:         "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
678:     )
679:     await conn.execute(
680:         "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width, height,
"
681:         "size_bytes, mime_type, matched, forwarded, seen_at) "
682:         "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')"
683:     )

```

```

--
789: CREATE TABLE users (
790:     id INTEGER PRIMARY KEY, phone TEXT,
791:     enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
792: );
793: CREATE TABLE channels (
794:     id INTEGER PRIMARY KEY AUTOINCREMENT,
795:     user_id INTEGER NOT NULL REFERENCES users(id),
796:     channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
797:     UNIQUE (user_id, channel_id)
798: );
799: INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z');
800: ""
801: async with aiosqlite.connect(db_path) as conn:
802:     await conn.executescript(legacy_schema)
803:     await conn.commit()
804:
805: await db.init_db(db_path)
806:
807: async with aiosqlite.connect(db_path) as conn:
808:     await conn.execute("PRAGMA foreign_keys=ON")
809:     # Create a NEW account that exists only in accounts, not users_deprecated.
810:     await conn.execute(
811:         "INSERT INTO accounts (id, telegram_id, provider, created_at) "
--
1128: CREATE TABLE users (
1129:     id INTEGER PRIMARY KEY, phone TEXT,
1130:     enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
1131: );
1132: CREATE TABLE channels (
1133:     id INTEGER PRIMARY KEY AUTOINCREMENT,
1134:     user_id INTEGER NOT NULL REFERENCES users(id),
1135:     channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
1136:     UNIQUE (user_id, channel_id)
1137: );
1138: CREATE TABLE videos (
1139:     id INTEGER PRIMARY KEY AUTOINCREMENT,
1140:     user_id INTEGER NOT NULL REFERENCES users(id),
1141:     channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
1142:     document_id INTEGER, duration_s INTEGER, width INTEGER, height INTEGER,
1143:     size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
1144:     forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
1145:     UNIQUE (user_id, channel_id, message_id)
1146: );
1147: CREATE TABLE filter_config (
1148:     user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
1149: );
1150: ""
1151: async with aiosqlite.connect(db_path) as conn:
1152:     await conn.executescript(legacy_schema)
1153:     await conn.execute(
1154:         "INSERT INTO users (id, phone, enabled, created_at) "
1155:         "VALUES (111, '+1', 1, '2024-01-01T00:00:00Z')"
1156:     )
1157:     # Two channels so message_id-only keys don't collide.
1158:     await conn.executemany(
1159:         "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
1160:         "VALUES (111, ?, ?, '2024-01-01T00:00:00Z')",
1161:         [(-1001, "Channel A"), (-1002, "Channel B")],
1162:     )
1163:     await conn.executemany(
1164:         "INSERT INTO videos (user_id, channel_id, message_id, document_id, "
1165:         "duration_s, width, height, size_bytes, mime_type, matched, forwarded, seen_at) "

```

```
--
1328: CREATE TABLE users (
1329:     id INTEGER PRIMARY KEY, phone TEXT,
1330:     enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
1331: );
1332: CREATE TABLE channels (
1333:     id INTEGER PRIMARY KEY AUTOINCREMENT,
1334:     user_id INTEGER NOT NULL REFERENCES users(id),
1335:     channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
1336:     UNIQUE (user_id, channel_id)
1337: );
1338: CREATE TABLE videos (
1339:     id INTEGER PRIMARY KEY AUTOINCREMENT,
1340:     user_id INTEGER NOT NULL REFERENCES users(id),
1341:     channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
1342:     document_id INTEGER, duration_s INTEGER, width INTEGER, height INTEGER,
1343:     size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
1344:     forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
1345:     UNIQUE (user_id, channel_id, message_id)
```

[Showing results with pagination = limit: 120]

18. assistant (2026-07-06T07:58:22.459Z)

Now I have a valid legacy schema. Let me rerun the repro with the correct columns.

19. user (2026-07-06T07:58:34.576Z)

```
WITHOUT filter_config: {'destinations': 0, 'routing_rules': 0, 'delivery_attempts': 0,
'content_items': 1, 'policies': 0}
WITH filter_config: {'destinations': 1, 'routing_rules': 1, 'delivery_attempts': 1,
'content_items': 1, 'policies': 1}
```

20. assistant (2026-07-06T07:58:47.303Z)

The repro confirms the mechanical behavior exactly as the finding describes: without a `filter\_config` row, an account with a channel + forwarded video gets `destinations=0, routing\_rules=0, delivery\_attempts=0` while `content\_items=1` is still populated. With the row, all get 1.

Now the verdict assessment. The mechanical defect is real and reproduces. But the finding is explicitly a "low/latent" note whose own evidence establishes the trigger is structurally unreachable:

1. `bootstrap.py:75` unconditionally calls `save\_filter\_config(conn, user.id, DEFAULT\_FILTER\_CONFIG)` for every onboarded account ? I confirmed this. So every real account has a `filter\_config` row.
2. An account cannot accumulate `channels`/forwarded `videos` without a running bootstrapped client, which requires having gone through onboarding.
3. Note that this also isn't purely a #37-diff issue: migration 003 (db.py:620-629, pre-existing on this branch from Track B/#35) has the *same* dependency ? no `filter\_config` row means no default `policy` either (repro shows `policies=0`). So the "routeless" state described is partly a consequence of the pre-existing policy backfill, not solely new code.

The defect described genuinely exists in the code as written (step 1 at db.py:702 and the subquery at db.py:720 are FROM `filter\_config\_deprecated`), and the wrong behavior reproduces. It is correctly characterized as latent and very-low-probability. The trigger is realistically unreachable given bootstrap always writes filter_config, but "structurally impossible" is a slight overstatement (a hand-edited/corrupted DB or a partially-failed bootstrap could produce it, and the migration does silently drop forwarded-delivery history in that case rather than erroring). This makes it CONFIRMED at low severity ? a hygiene/robustness gap, not a real-world data-loss bug.

21. user (2026-07-06T07:58:56.332Z)

Structured output provided successfully